Texas College of Management & IT

Bachelor of Information Technology (BIT)

Introduction to Flutter & Dart

Submitted By:
Name: Govinda Kumar Rajbanshi
LCID: LC00017001924
Semester: 6th Sem
Section: A

Submitted To:
Instructor Name: Ashish Gautam
Course: Mobile App Development using Flutter (BIT 244)
Date of Submission: 26th July, 2025

Introduction to Flutter & Dart

Table of Contents

Task 1: State Management (30 Marks) 3
Compare Provider, BLoC, and Riverpod with use-case examples. 3
Explain the role of BuildContext in state management. 4
Task 2: API & Firebase Integration (30 Marks) 4
Explain REST API concepts (GET/POST) and JSON parsing. 4
Describe Firebase Auth/Firestore security rules. 6

Assignment Task (Theory Portion)

Task 1: State Management (30 Marks)

Compare Provider, BLoC, and Riverpod with use-case examples.

Provider is the most widely taught and used state management tool when starting with Flutter. It wraps around Flutter's inherited widget system, allowing widgets to "listen" to changes in shared state and rebuild accordingly. Its simplicity makes it approachable for beginners or for lightweight apps that don't require heavy logic separation. For example, imagine you're building a basic shopping cart app. With Provider, you might use `ChangeNotifier` to hold cart items and notify listeners whenever a product is added or removed. This allows for reactive updates across your app without overly complex wiring. However, Provider can become cumbersome as your app scales—especially when multiple models and dependencies start interacting. That's when its verbose syntax and tight coupling to the widget tree show limitations.

BLoC (Business Logic Component), on the other hand, takes a more structured and reactive approach. It promotes a separation of concerns, keeping your business logic completely distinct from your user interface. BLoC heavily utilizes streams—events are added into a `Sink`, and new state flows out through a `Stream`. This design is especially beneficial for applications requiring complex data handling, multiple user interactions, and realtime updates. For instance, a live chat app where messages are streamed in and out, with typing indicators and reactions, is an ideal use-case for BLoC. Yes, it demands more boilerplate, and the initial learning curve is steeper, but you get a testable, scalable, and robust architecture, especially appreciated in enterprise-level applications.

Riverpod emerges as a modern evolution of Provider. It addresses many of Provider's limitations by making state management more independent from the widget tree. Riverpod doesn't rely on `BuildContext`, so you can read and manipulate your providers anywhere—even in business logic layers, background services, or async flows. This makes it particularly suitable for large-scale apps or those requiring cleaner architecture. Take, for example, a weather app where temperature data is fetched asynchronously, and different widgets react to changes without the fragility of tree-based dependency chains. Riverpod's composability and robust support for asynchronous programming make this a breeze. Moreover, its compile-time safety and IDE tooling help prevent subtle bugs that can creep into Provider-based implementations.

Each of these tools has its place. Provider is excellent for apps with straightforward state interactions. BLoC excels when the logic is complex and the need for strict separation is high. Riverpod is often the best fit when you want flexibility, testability, and modern patterns without excessive boilerplate. Ultimately, the choice depends on the size, complexity, and goals of your app—and your personal preference for code structure.

[3]

Explain the role of BuildContext in state management.

In Flutter, `BuildContext` plays a foundational role not just in building the widget tree but also in accessing and managing state. Think of it as a handle or reference to the location of a widget in the widget tree. Through this handle, widgets can communicate with their ancestors and fetch data or dependencies provided higher up. In the context of state management, especially when using tools like `Provider`, `BuildContext` becomes your bridge to tap into shared state objects. For example, using `Provider.of<T>(context)` or `context.watch<T>()`, you're essentially instructing Flutter to walk up the widget tree, locate a provider of type `T`, and return its value. This implies that state management solutions tied to the widget tree are deeply dependent on where and how `BuildContext` is used.

This dependency introduces both strengths and limitations. On the positive side, it allows for localized and highly reactive UIs—when state changes, only widgets with relevant contexts rebuild. However, it also means state access is constrained by the tree structure. If you try to access a provider before it's initialized or outside its scope, you'll run into runtime errors. This is one of the reasons why newer tools like Riverpod aim to decouple state management from `BuildContext` entirely. Riverpod allows you to read and manipulate providers from anywhere, not just inside widget build methods. This promotes cleaner architecture and better separation of concerns, especially in complex apps with business logic and background services.

Another aspect where `BuildContext` shows its influence is lifecycle and rebuild behavior. Since each widget has its own context, managing state updates in nested widgets requires careful handling of context references—using the wrong one might lead to unexpected UI behavior or even crashes. Tools like `Consumer`, `Selector`, or `BlocBuilder` manage this for you by scoping context usage tightly and reacting only when necessary. In summary, `BuildContext` is pivotal in bridging widgets and state, especially when using tree-bound management systems, but it also illustrates why architectural choices—like whether to rely on or break free from context—matter immensely in Flutter development.

Task 2: API & Firebase Integration (30 Marks)

Explain REST API concepts (GET/POST) and JSON parsing.

REST (Representational State Transfer) is an architectural style widely used for building web services. It revolves around the idea that a client (like a mobile app or frontend) interacts with a server through stateless requests, typically over HTTP. The server provides access to resources—like user profiles, posts, or products—which are represented by URLs. Think of a REST API as a set of endpoints that expose data or services. These endpoints follow predictable patterns, such as `/users`, `/products`, or `/posts`, and can perform operations using HTTP methods like GET, POST, PUT, DELETE, etc.

Two of the most commonly used HTTP methods in REST are **GET** and **POST**. A `GET` request is used to retrieve data from the server without modifying anything. For example, when an app wants to display a list of articles, it might send a GET request to `https://api.example.com/articles`, and the server will respond with the data for those articles. On the other hand, a `POST` request is used to send data to the server, often to create a new resource. For instance, when a user submits a new comment on a post, the app might send a POST request to `https://api.example.com/comments` with the comment text in the request body. The server processes the data, stores it, and typically returns a confirmation or the created resource.

Now, to facilitate communication between clients and servers, data is often exchanged in **JSON (JavaScript Object Notation)** format. JSON is lightweight, readable, and easy to parse, making it ideal for representing structured data. For example, the response from a GET request might look like this:

```
{
  "id": 101,
  "title": "Introduction to Flutter",
  "author": "Jane Doe",
  "published": true
}
```

When this JSON arrives at the client (say, your Flutter app), it needs to be parsed into native objects to be useful. Parsing means converting the JSON string into data types your code understands—like maps, lists, or custom models. In Flutter (with Dart), this might involve using `json.decode()` to turn the raw string into a `Map`, and then mapping that to a class like `Article`. You can manually write the mapping logic or use packages like `json_serializable` to automate it.

In essence, REST APIs form the backbone of dynamic apps, enabling features like user login, data updates, or live content fetching. GET and POST are foundational to how these interactions work, and JSON parsing transforms the abstract data into tangible, usable information in your app's UI. This model ensures modularity, scalability, and a clean separation between frontend and backend logic—an indispensable pattern in modern development.

Describe Firebase Auth/Firestore security rules.

Firebase Authentication and Firestore security rules work hand-in-hand to provide secure, scalable access control for mobile and web apps. Firebase Authentication handles the identity side of things, allowing users to sign up, sign in, and verify their identity via email/password, phone, or third-party providers like Google, Facebook, or Apple. Once authenticated, Firebase issues a secure token for the user, which is passed along with any request to Firestore. Firestore's security rules then use this token to make real-time access decisions about what users can read, write, or update in the database.

Firestore security rules are essentially conditional statements written in a special syntax that evaluates each database request. These rules determine who can access which documents or collections, under what circumstances, and how data is validated. For example, you can write a rule that allows users to only access their own profiles by checking if the document ID matches the `request.auth.uid`. This keeps sensitive user-specific data protected without needing to build a custom backend. Rules can also enforce field-level validation, such as ensuring a `createdAt` timestamp is never overwritten or verifying that a score value falls within an acceptable range. Because these rules run on every read/write request, they must be precise and performant to avoid unintended access or bottlenecks.

A core principle of Firestore rules is that they are declarative and context-aware, meaning they rely heavily on the `request` object, which includes the user's authentication state, the operation type (get, list, update, etc.), and the resource being accessed. It's possible to structure rules to handle different access levels—for example, allowing general reads but restricting writes to only admins. You can even define reusable rule functions for modular, scalable policies across collections.

Altogether, Firebase Auth and Firestore rules offer a powerful way to build secure apps without running your own server. But they also require careful design and regular testing using Firebase's built-in Rules Simulator or unit tests, as misconfigured rules can inadvertently expose or restrict vital data.